



Maschinelles Lernen

4. Klassifizierung und logistische Regression

Prof. Dr. Dennis Janka

Lernziele

- Klassifizierungsprobleme von Regressionsproblemen abgrenzen
- Logistisches Regressionsmodell und Sigmoid-Funktion verstehen
- Entscheidungsgrenze interpretieren
- Log-Loss als Verlustfunktion für binäre Klassifizierung kennen
- Metriken zur Modellbewertung anwenden (Accuracy, Precision, Recall)
- Nichtlineare Entscheidungsgrenzen durch Feature-Engineering erzeugen



Klassifizierungsprobleme



Was ist Klassifizierung?

Regressionsproblem: Vorhersage einer numerischen Zielvariable

Klassifizierungsproblem: Vorhersage einer *kategoriellen* Zielvariable

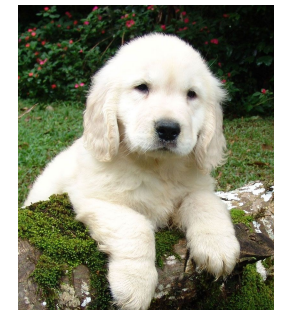
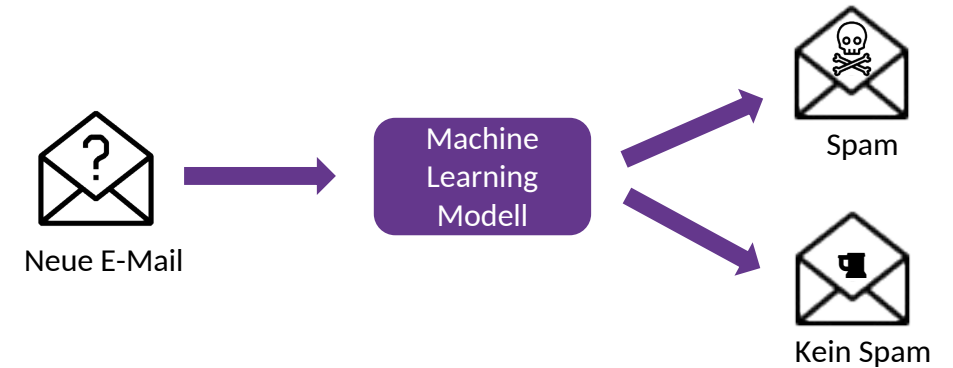
- Eingabe: Featurevektor $\mathbf{x} \in \mathbb{R}^p$
- Ausgabe: eine von K diskreten Klassen $C_1, \dots, C_K$

Klassifizierung



Als Klassifizierung (“Classification”) werden Vorhersagen bezeichnet bei denen das Label **kategorisch** ist. Beispiele:

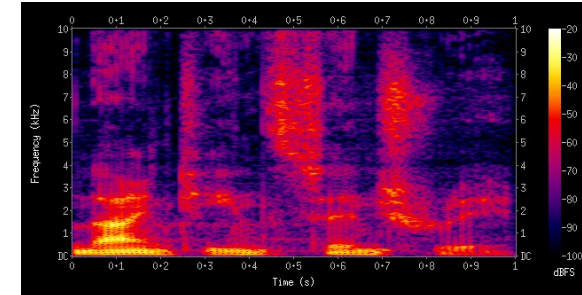
- Spam-Filter
x: Vorkommen bestimmter Wörter, Headerdaten
y: „Spam“ oder „kein Spam“
- Bilderkennung: Was ist auf dem Bild?
x: Alle Pixel des Bilds
y: Objekt, z.B. „Katze“, „Hund“, „Maus“, ...



Klassifizierung



- Betrugserkennung: Ist diese Kreditkartentransaktion okay?
x: Ort, Betrag, Verkäufer, vergangene Transaktionen, ...
y: „Betrug“ oder „legitim“
- Spracherkennung: Was hat die Person gesagt?
x: Audiospektrogramm
y: Wörter



Sehr viele Probleme aus dem Bereich KI (maschinelles Sehen, Verarbeitung natürlicher Sprache) sind im Kern Klassifikationsprobleme (obwohl man sie manchmal nicht direkt als solche erkennt)

Kodierung des Labels



Zunächst: binärer Fall, also $K = 2$

Label wird als 0 / 1 kodiert:

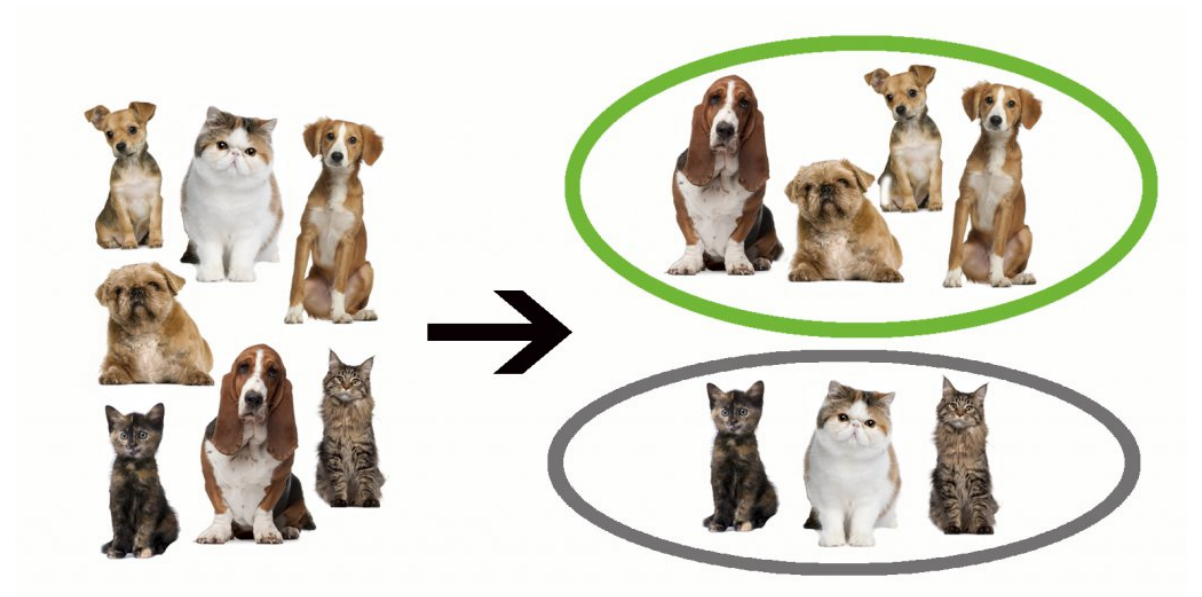
Beispiele:

Betrug / legitim:

$y = 0$: legitim; $y = 1$: Betrug

Hund oder Katze:

$y = 0$: Hund; $y = 1$: Katze





Logistisches Regressionsmodell



Default-Datensatz

Anwendungsfall: Vorhersage von Kreditkarten-Zahlungsausfällen

Features:

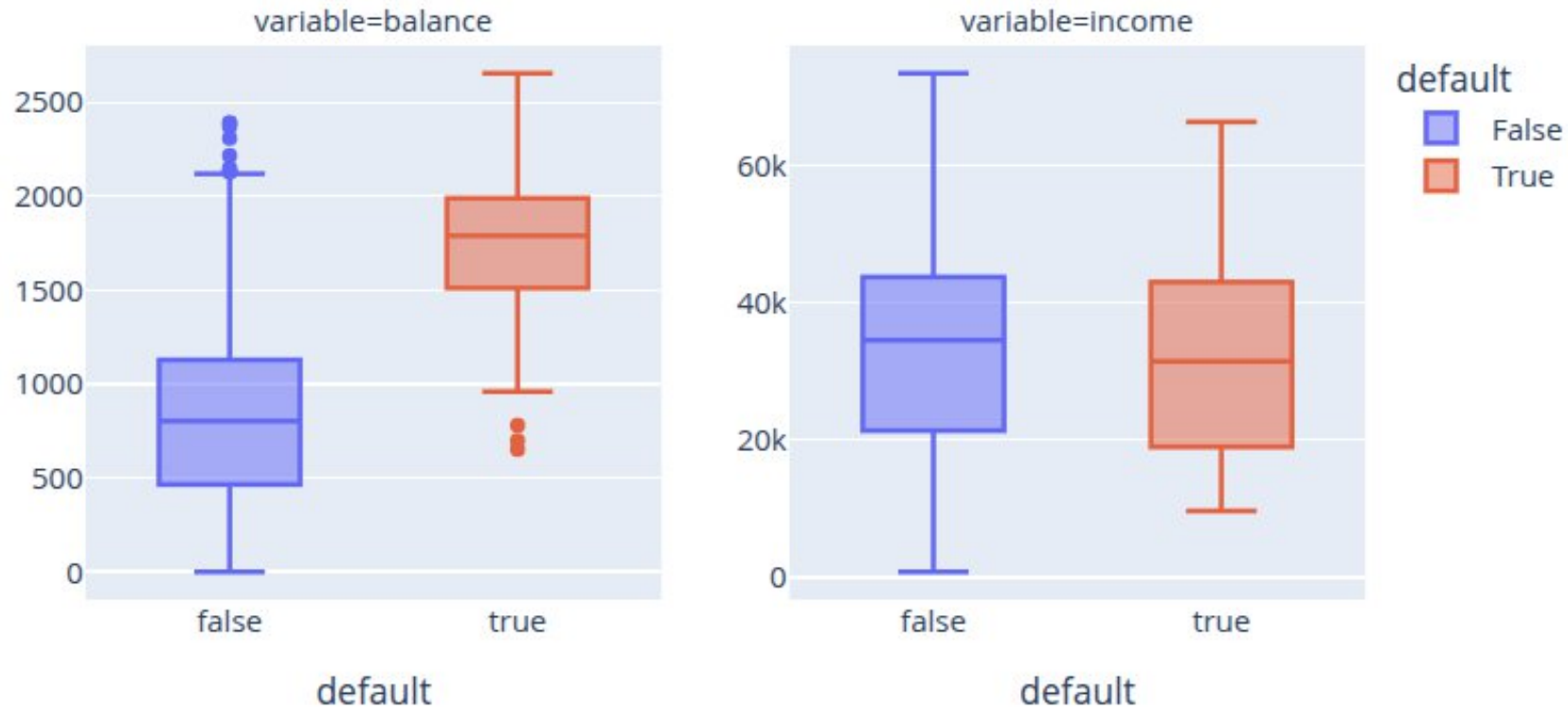
- balance: ausstehender Betrag auf dem Kreditkartenkonto
- income: Jahreseinkommen
- student: studiert die Person noch?

Zielvariable default:

- 1 = Zahlungsausfall
- 0 = kein Zahlungsausfall

Zunächst: Nur ein Feature

Datenanalyse



Welches ist besser?

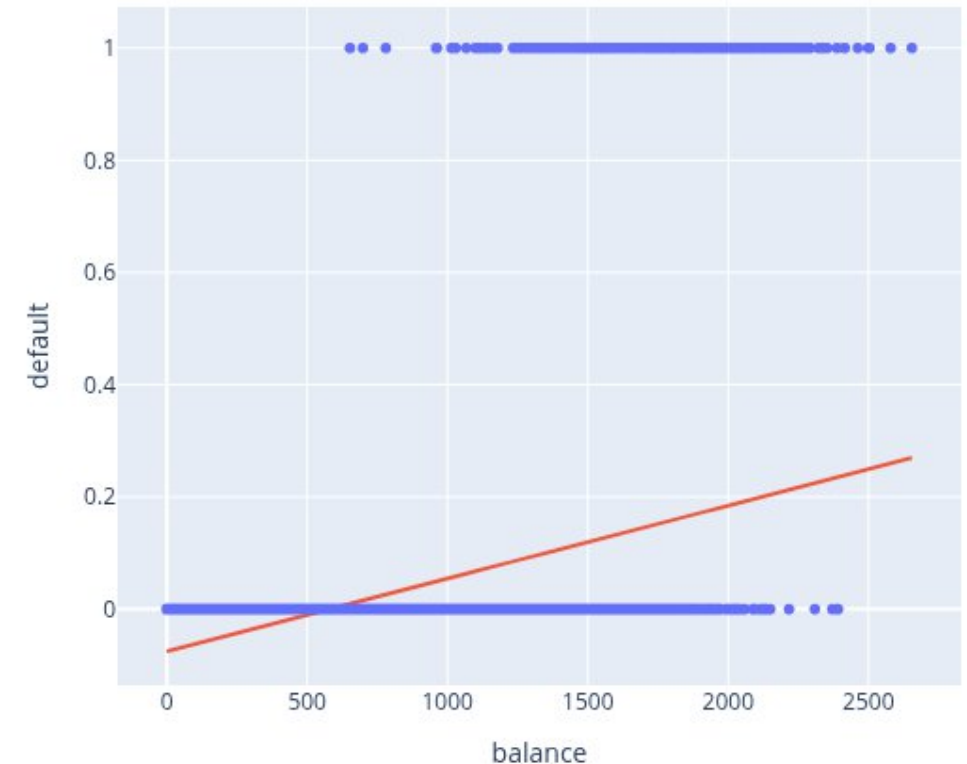
Lineare Regression für Klassifizierung?

Kodierung: default = 0 oder 1

Trainiere lineare Regression: $f(x) = w_0 + w_1x$

Probleme:

- Vorhersagen liegen außerhalb $[0,1]$
- Keine Interpretation als Wahrscheinlichkeit möglich

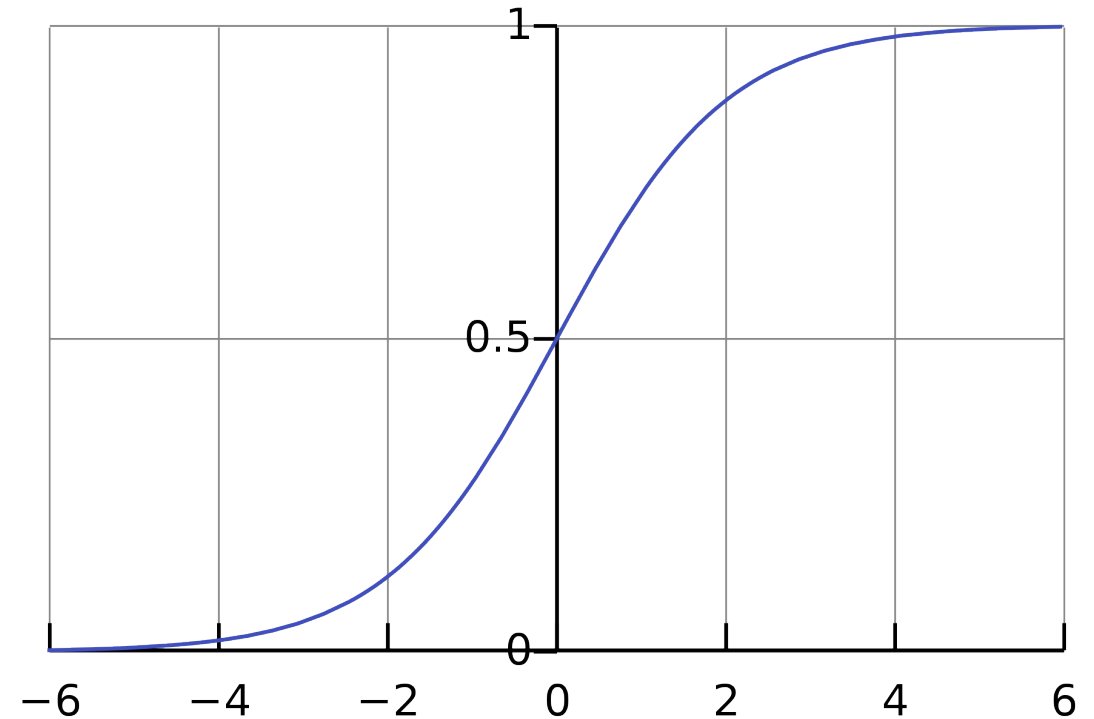


Sigmoid-Funktion



Um sicherzustellen, dass $f(x) \in [0,1]$, wird die lineare Funktion durch die **Sigmoid-Funktion** σ "gestaucht":

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



$$\lim_{x \rightarrow -\infty} \sigma(z) = 0$$

$$\lim_{x \rightarrow \infty} \sigma(z) = 1$$

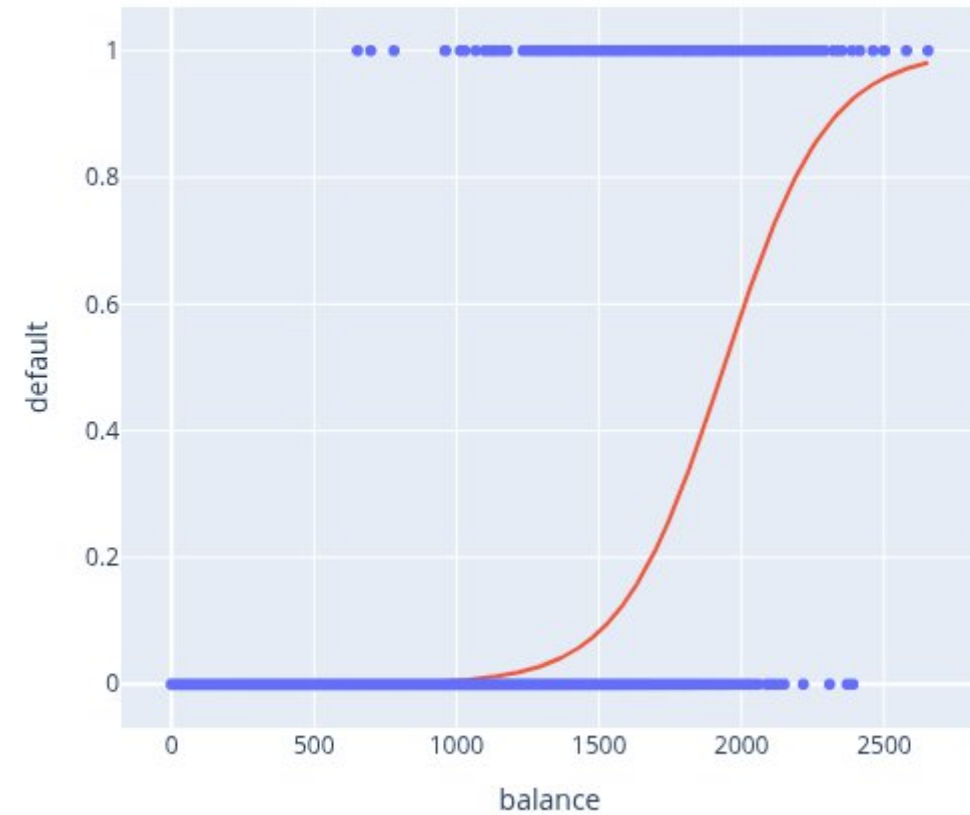
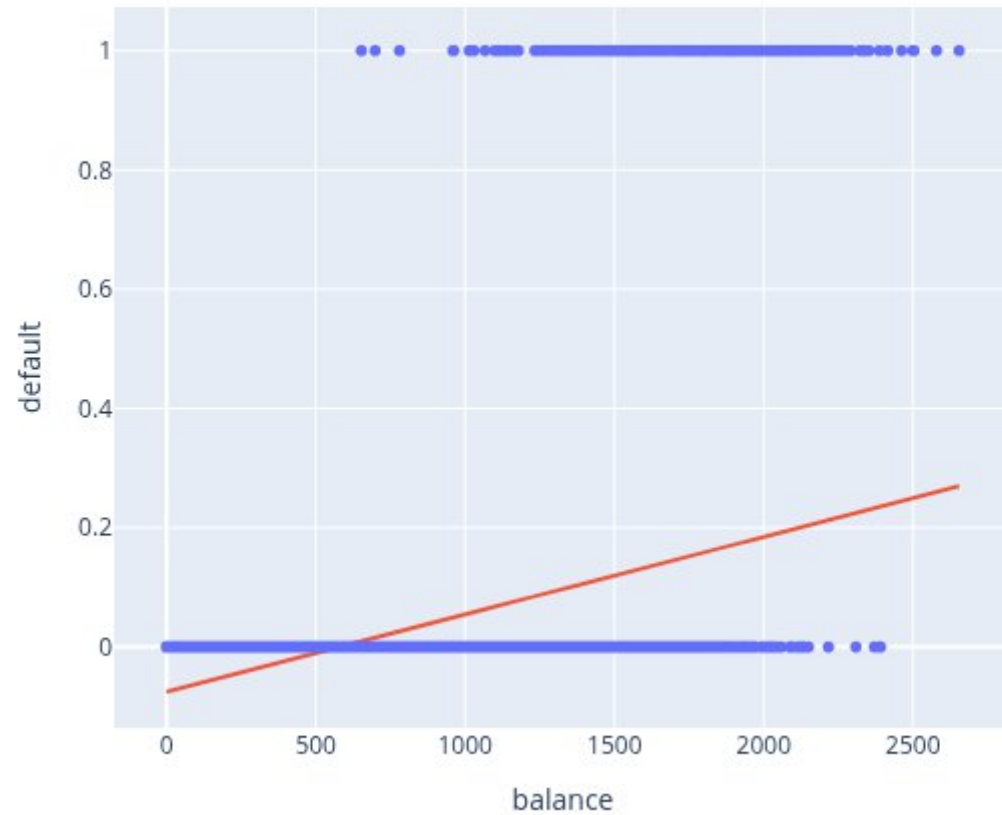
Sigmoid-Funktion als Modellfunktion

Für ein Feature:

- $f(x) = \sigma(w_0 + w_1 x) = \frac{1}{1 + e^{-(w_0 + w_1 x)}}$
- w_0 und w_1 werden durch Training bestimmt, x ist das Feature des Datenpunkts

Die Vorhersage wird als Wahrscheinlichkeit interpretiert, dass der Punkt zur Klasse 1 gehört

Trainiertes lineares Modell

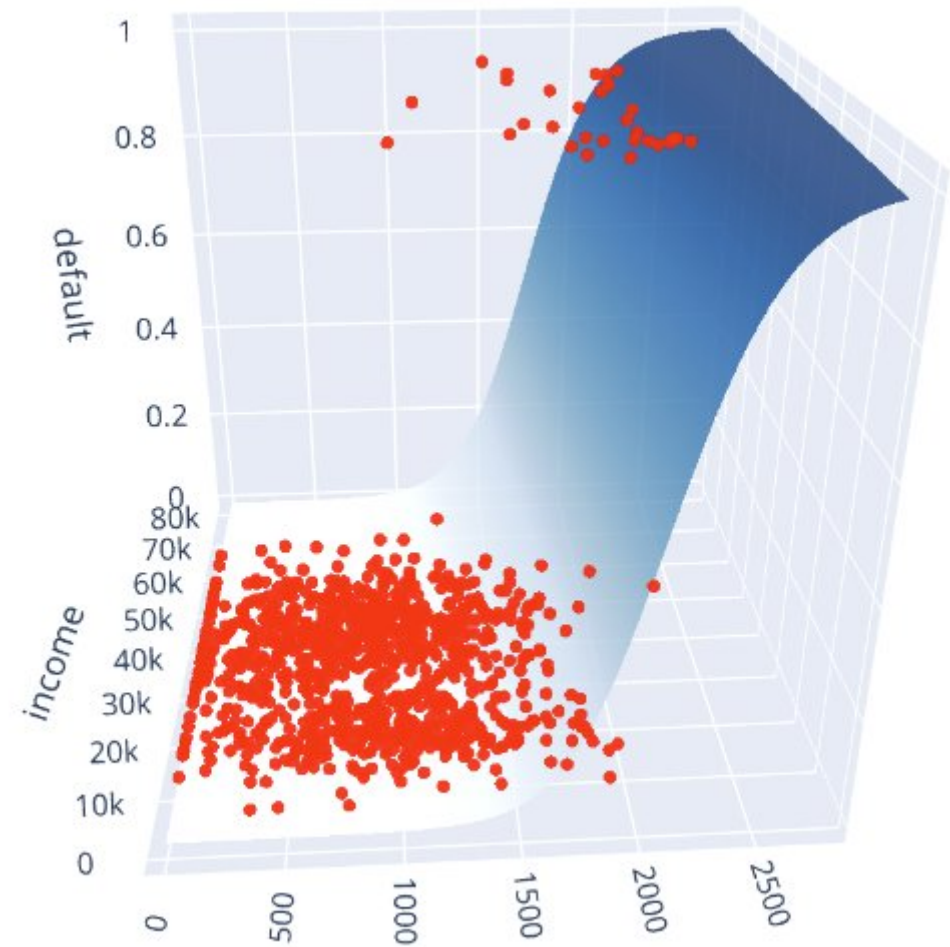


Logistisches Modell für mehrere Features



Modellfunktion für mehr als ein Feature:

$$f(\mathbf{x}) = \frac{1}{1 + e^{-(w_0 + w_1x_1 + w_2x_2 + \dots + w_px_p)}}$$



Vorhersage der Klassenzugehörigkeit

In vielen Fällen will man anstelle einer Wahrscheinlichkeit eine konkrete Klasse als Output der Vorhersage (z.B. “credit default” oder “not credit default”).

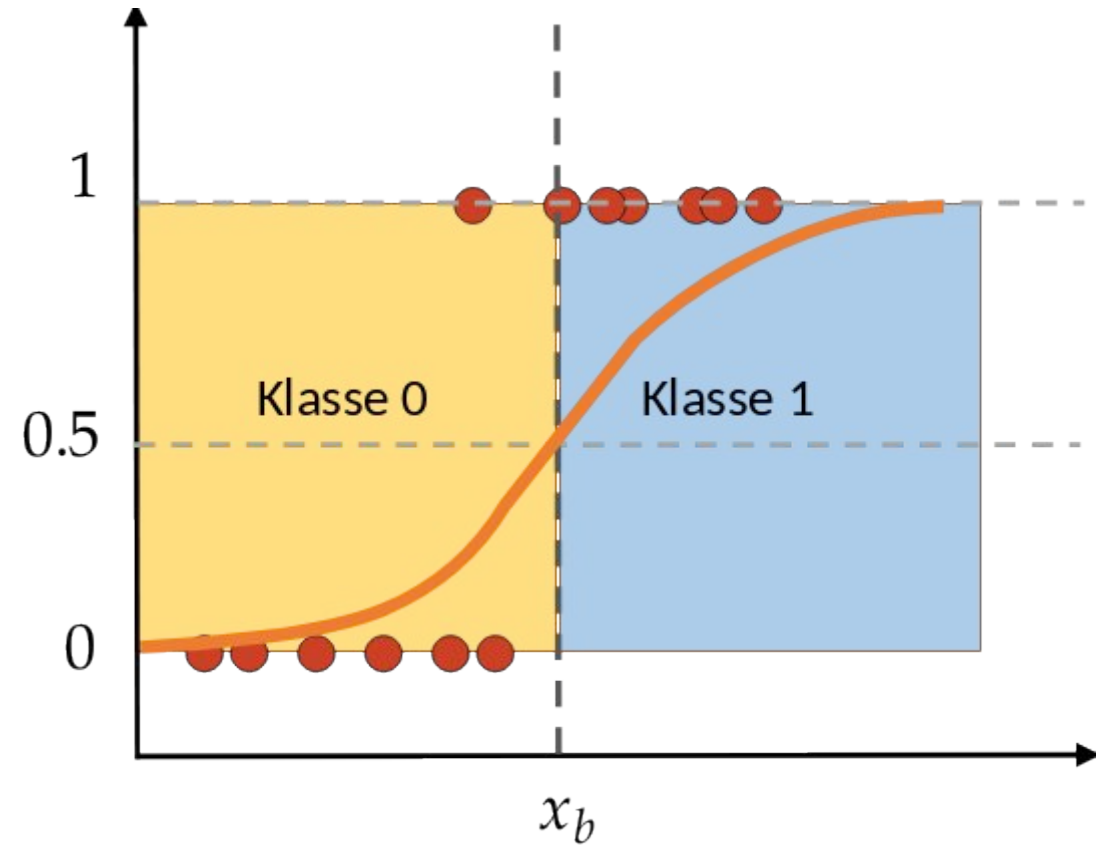
Lösung: Man legt einen Schwellwert B fest (default: 0.5) und testet, ob $f(\mathbf{x})$ (also die W'keit) über oder unter diesem Schwellwert liegt:

$$Klasse(\mathbf{x}) = \begin{cases} 0, & \text{falls } f(\mathbf{x}) < B \\ 1, & \text{falls } f(\mathbf{x}) \geq B \end{cases}$$

Beispiel: $f(\mathbf{x}) = 0.894 \rightarrow \mathbf{x}$ ist “credit default”

Entscheidungsgrenze

- Es gibt Punkte $\mathbf{x}_B$ für die gilt
$$f(\mathbf{x}_B) = B.$$
- Diese Punkte werden als **Entscheidungsgrenze** bezeichnet.
- Sie teilen den Feature Raum in 2 Teile entsprechend der Klassifikation in die Klassen 0 und 1
- Für ein Feature gibt es genau einen Punkt für den das zutrifft.

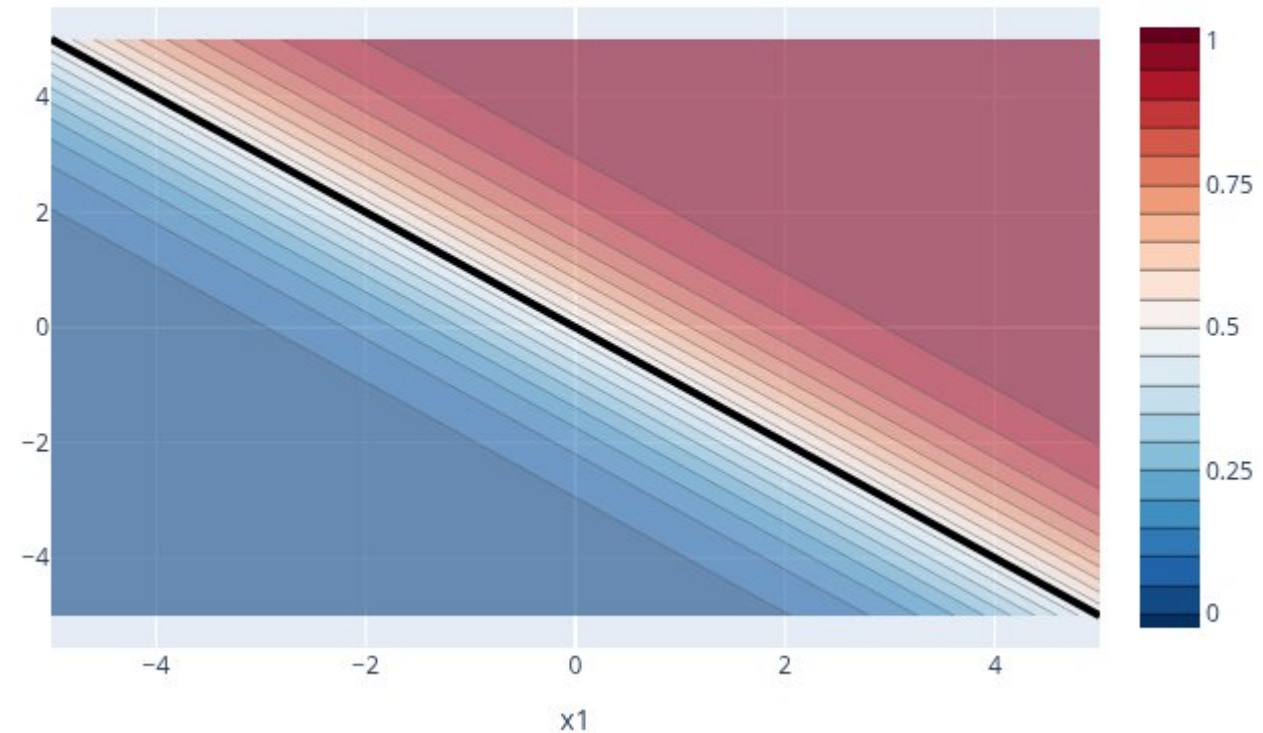


Entscheidungsgrenze

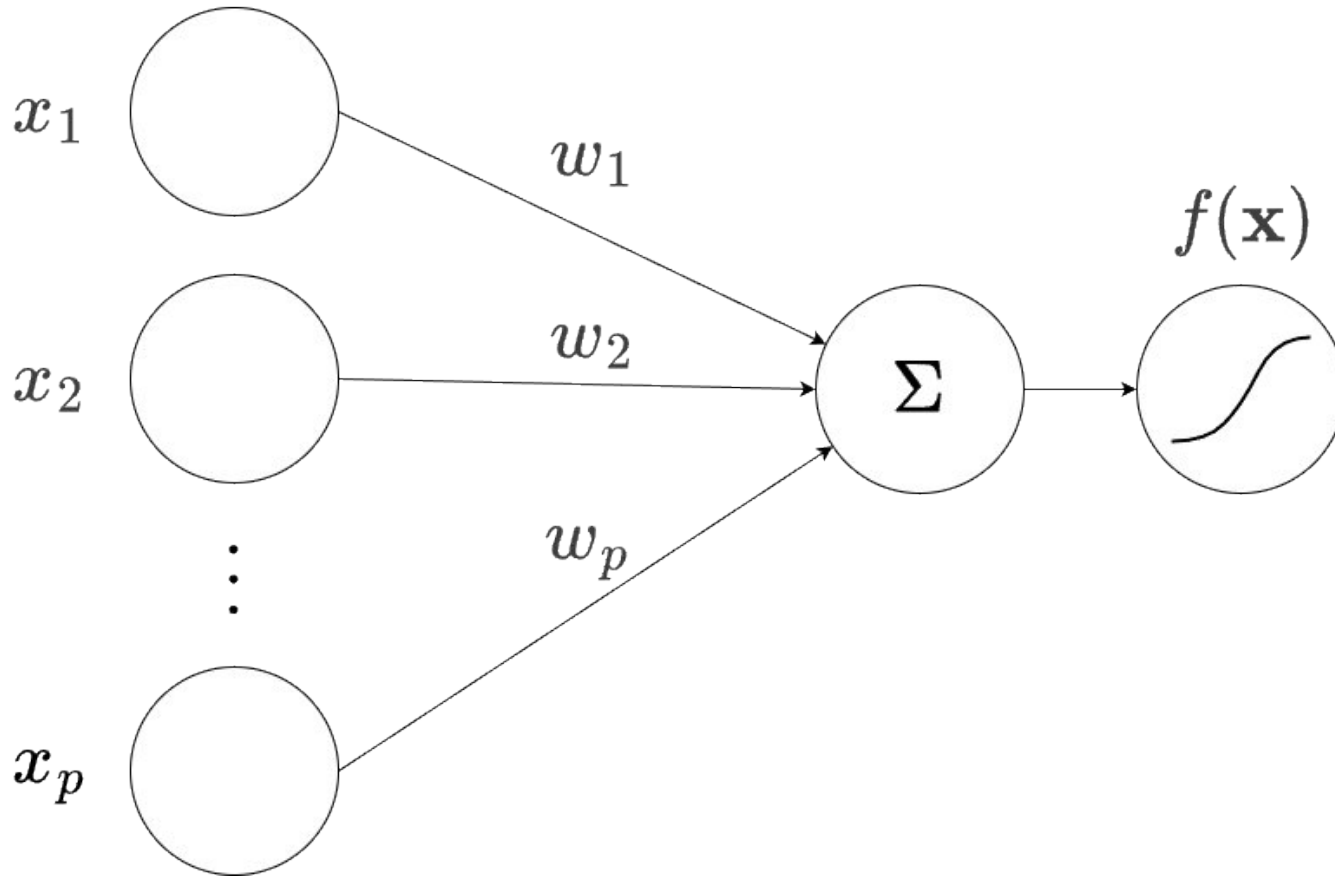
Im zweidimensionalen Featureraum (d.h. es gibt zwei Features x_1 und x_2), ist diese Grenze eine **Gerade**.

→ **Logistische Regression ist ein lineares Modell**

Featureraum	Entscheidungsgrenze
1D ($p = 1$)	ein Punkt
2D ($p = 2$)	eine Gerade
3D ($p = 3$)	eine Ebene
p D	Hyperebene



Darstellung als neuronales Netz



Visualisierung der
Rechenoperationen:
Gewichtete Summation +
Sigmoid („Aktivierung“)



Modelltraining: Log-Loss



Warum nicht MSE?

Bei binärer Klassifizierung: Label $y \in \{0,1\}$, Vorhersage $\hat{y} \in (0,1)$

MSE ist beschränkt: maximaler MSE bei schlechtester Vorhersage ist $(1-0)^2 = 1$

→ Zu schwacher Anreiz, Fehler zu korrigieren



Idee Log-Loss

Bessere Idee: Loss-Funktion, die bei schlechten Vorhersagen *beliebig groß* werden kann:

- Wenn $y = 1$: Loss = $-\log(\hat{y})$ — groß wenn $\hat{y}$ klein
- Wenn $y = 0$: Loss = $-\log(1-\hat{y})$ — groß wenn $\hat{y}$ groß



Log-Loss (Binäre Kreuzentropie)

Für einen einzelnen Datenpunkt:

$$\ell(\mathbf{w}) = -(y \log(\hat{y}) + (1-y) \log(1-\hat{y}))$$

Loss-Funktion über alle N Trainingsdaten (Mittelwert):

$$L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

mit $\hat{y}_i = \sigma(\mathbf{w}^T \mathbf{x}_i)$

Minimierung der Log-Loss Funktion

Optimierungsproblem:

$$\min_{\mathbf{w}} L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\sigma(\mathbf{w}^T \mathbf{x}_i)) + (1-y_i) \log(1-\sigma(\mathbf{w}^T \mathbf{x}_i))]$$

Nicht analytisch lösbar → **Gradientenabstieg**

Starte mit Schätzung $\mathbf{w}^{[0]}$ und iteriere für $k = 0, 1, 2, \dots$:

$$\mathbf{w}^{[k+1]} := \mathbf{w}^{[k]} - \alpha^{[k]} \nabla L(\mathbf{w})^T$$

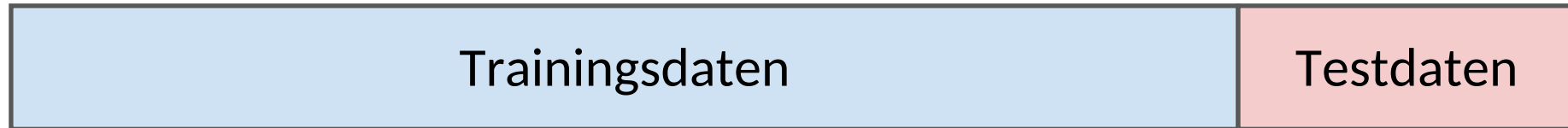


Evaluation von Klassifizierungsmodellen



Evaluation des Modells

- Um zu Testen wie zuverlässig das trainierte Modell ist, wird die logistische Regression auf die Testdaten angewandt (analog zu lineare Regression).



- Im Gegensatz zur linearen Regression evaluiert man bei der Klassifikation nicht analog zur Kostenfunktion (R^2 , (root)MSE), sondern über die **Klassenzugehörigkeit**.
- Daraus ergeben sich eine Reihe von Evaluierungsmetriken, unter anderem:
 - Accuracy
 - Precision und Recall
 - False positive rate

Konfusionsmatrix

Vorhersagen eines binären Klassifikators werden kategorisiert als:

	Echtes Label 1	Echtes Label 0
Vorhersage 1	True Positive (TP)	False Positive (FP)
Vorhersage 0	False Negative (FN)	True Negative (TN)

Entscheidungsfälle

Beispiel: Wir nutzen logistische Regression um vorherzusagen ob eine Kreditkartentransaktion Betrug ist (1) oder okay ist (0).

Welcher Entscheidungsfall liegt vor?

1. Das Modell markiert eine Transaktion als Betrug, obwohl die Transaktion okay ist.
2. Das Modell markiert eine Transaktion als Betrug, welche sich als Betrug herausstellt.
3. Das Modell markiert eine Transaktion als okay, welche sich als Betrug herausstellt.

Visualisierung: Konfusionsmatrix

	Label 1	Label 0	
Vorhersage 1	True positive (N_{TP})	False positive (N_{FP}) (Fehler 1. Art)	Σ : Alle positiven Vorhersagen
Vorhersage 0	False negative (N_{FN}) (Fehler 2. Art)	True negative (N_{TN})	Σ : Alle negativen Vorhersagen
	Σ : Alle positiven Labels	Σ : Alle negativen Labels	

$$N = N_{TP} + N_{TN} + N_{FP} + N_{FN}$$

Evaluationsmetriken

$$\text{Accuracy} = \frac{N_{TP} + N_{TN}}{N}$$

Anteil aller korrekten Vorhersagen.

$$\text{Precision} = \frac{N_{TP}}{N_{TP} + N_{FP}}$$

Anteil korrekt positiver Vorhersagen an *allen positiven* Vorhersagen.

Evaluationsmetriken

$$\text{True positive rate (recall)} = \frac{N_{TP}}{N_{TP} + N_{FN}}$$

Anteil korrekt erkannter positiver Fälle an *allen tatsächlich positiven* Fällen.

$$\text{False Positive Rate} = \frac{N_{FP}}{N_{FP} + N_{TN}}$$

Anteil der tatsächlich negativen Fälle, die das Modell falsch vorhersagt.

Alle Metriken liegen zwischen 0 und 1.

Ziel: hohe Accuracy, Precision und Recall, niedrige False Positive Rate.

Welche Metrik ist wichtig?

Die richtige Wahl hängt vom Anwendungsfall ab

Leitfragen:

- Wie teuer sind false positives / false negatives?
- Wie wertvoll sind true positives / true negatives?

Beispiel 1: Medizinischer Test (krank / gesund):

- Hohes Risiko durch verpasste Kranke → Recall maximieren
- Viele false positives (gesund → krank) belastet das System

Beispiel 2: Chemieanlage (kritisch / unkritisch):

- Trivialklassifikator “nie kritisch” hat Accuracy $\approx 99.9\%$
- → Accuracy als alleinige Metrik hier ungeeignet!

Was ist ein guter Wert für eine Metrik?

Zwar liegen die Werte der Metriken zwischen 0 und 1 (wobei 1 bestmöglich und 0 schlechtestmöglich) aber welcher Wert dazwischen gut ist (und welcher u.U. trivial erreicht werden kann), ist nicht immer sofort offensichtlich und hängt vom Datensatz (und natürlich vom Anwendungsfall) ab.

	Label 1	Label 0
Vorhersage 1	True positive (TP)	False positive (FP)
Vorhersage 0	False negative (FN)	True negative (TN)

Bsp: $precision = \frac{TP}{TP+FP}$ hängt von der Verteilung der Labels ab, $recall = \frac{TP}{TP+FN}$
nicht.



Multi-Klassen Klassifizierung



Von binär zu K Klassen

Binäre Klassifizierung: $f: \mathbb{R}^p \rightarrow \mathbb{R}$ (eine Zahl zwischen 0 und 1)

Multi-Klassen Klassifizierung: $f: \mathbb{R}^p \rightarrow \mathbb{R}^K$

Für jede Klasse C_k wird eine Wahrscheinlichkeit ausgegeben:

$$f(\mathbf{x}) = \begin{pmatrix} P(y = C_1 \mid \mathbf{x}) \\ P(y = C_2 \mid \mathbf{x}) \\ \vdots \\ P(y = C_K \mid \mathbf{x}) \end{pmatrix}$$

Label wird **one-hot** kodiert: $(0, \dots, 1, \dots, 0)^T$

Softmax-Regression

K lineare Modelle berechnen einen Score pro Klasse:

$$\mathbf{z} = \mathbf{W}^T \mathbf{x} \in \mathbb{R}^K, \quad \mathbf{W} \in \mathbb{R}^{p \times K}$$

Softmax-Funktion wandelt Scores in Wahrscheinlichkeiten um:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

Eigenschaften: - Alle Werte liegen in $(0,1)$ - Summe aller Werte = 1 - Reihenfolge der Scores bleibt erhalten

Softmax-Regression: Training und Vorhersage

Vorhersage:

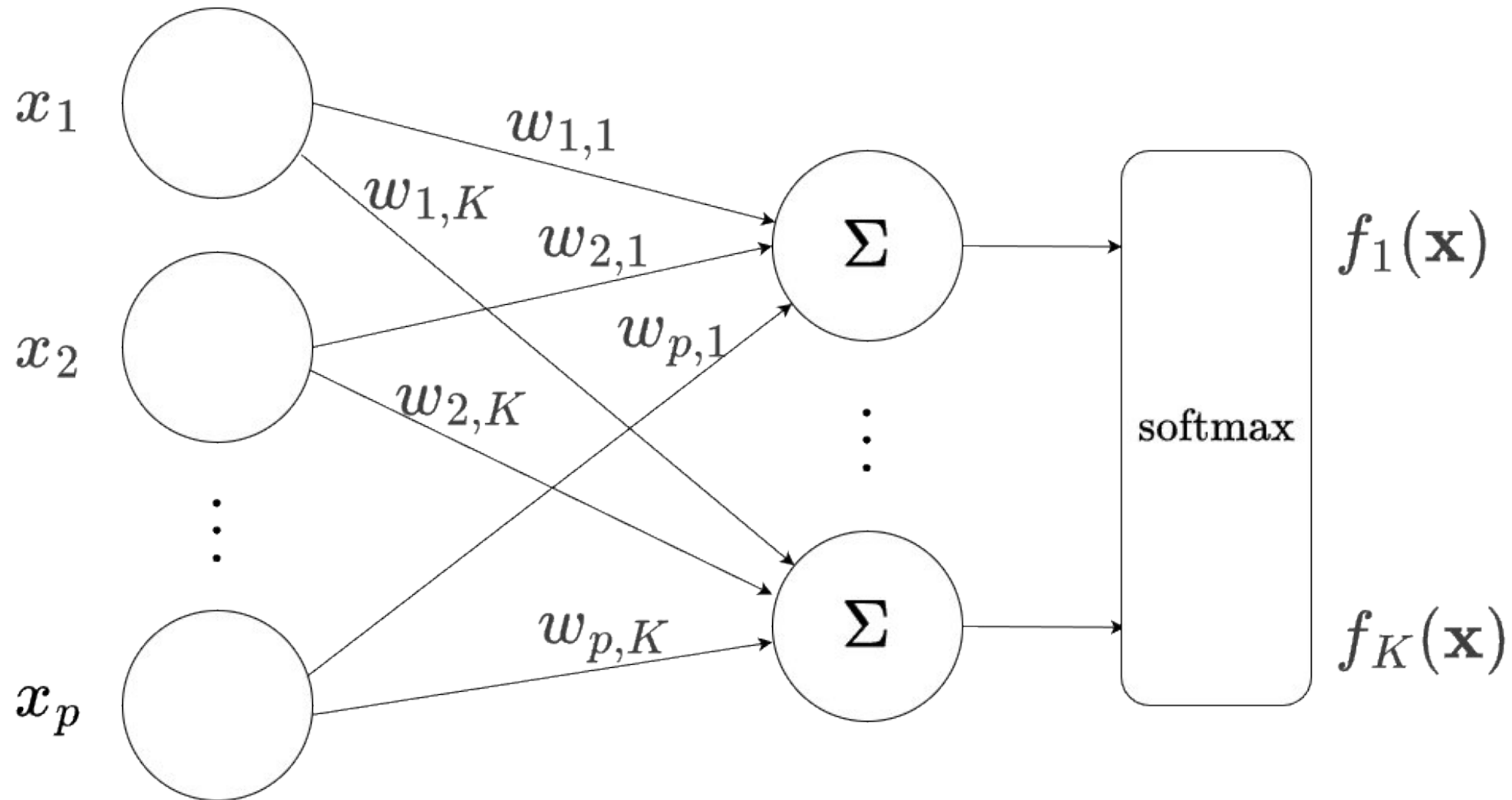
1. Scores berechnen: $\mathbf{z} = \mathbf{W}^T \mathbf{x}$
2. Wahrscheinlichkeiten: $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$
3. Klasse: $\hat{y} = \text{argmax}_k \hat{y}_k$ (= Klasse mit höchstem Score)

Training — Kreuzentropie-Verlust für K Klassen:

$$L(\mathbf{W}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K (y_i)_k \log(\hat{y}_i)_k$$

→ Minimierung wieder mit Gradientenabstieg

Darstellung als neuronales Netz



Confusion Matrix für K Klassen

Beispiel: Wettervorhersage (Regen / Schnee / Sonne)

Accuracy = Summe der Diagonale / N

Precision und Recall können **klassenweise** berechnet werden:

$$\text{Recall("Schnee")} = \frac{90}{10 + 20 + 90} = 0.75$$

$$\text{Recall("Sonne")} = \frac{60}{10 + 60 + 5} = 0.80$$

Vorhersage	Label		
	"Regen"	"Sonne"	"Schnee"
	80	10	10
	2	60	20
	5	5	90



Nichtlineare Features

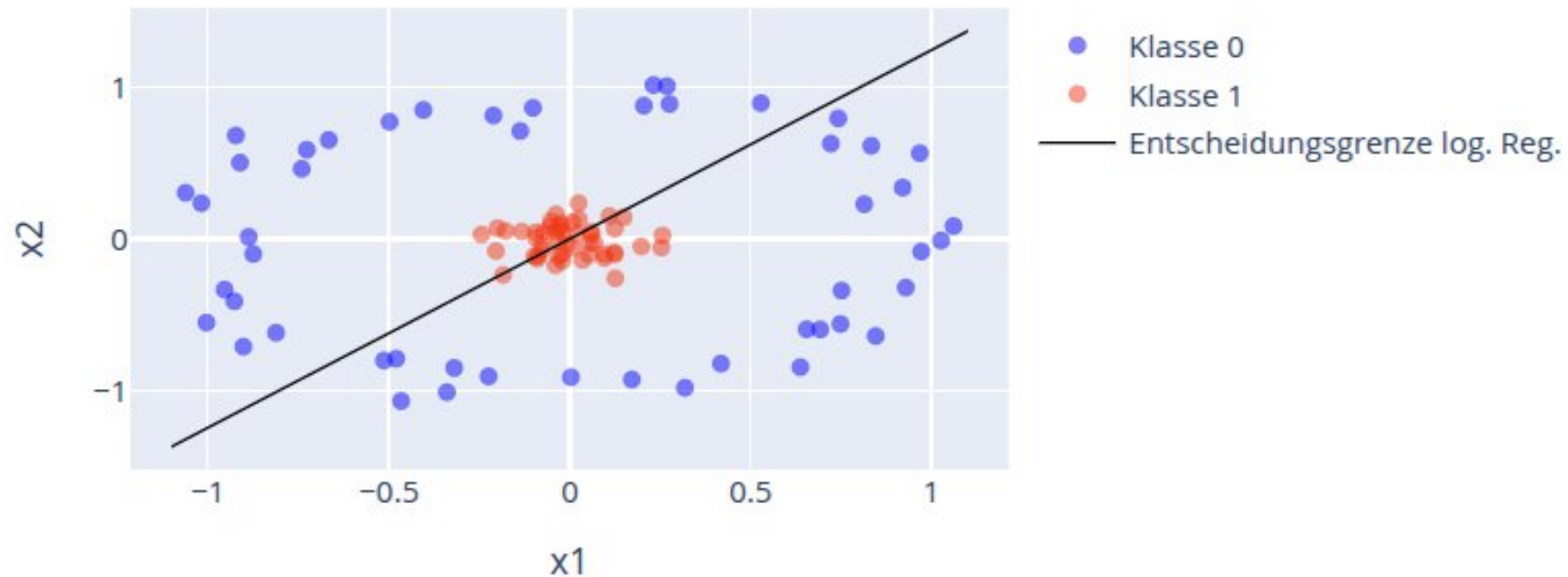


Grenzen der logistischen Regression

Die logistische Regression kann nur **lineare** Entscheidungsgrenzen lernen.

Problem: Manche Datensätze sind nicht linear trennbar.

Lösung: Feature-Engineering — neue Features aus bestehenden erzeugen

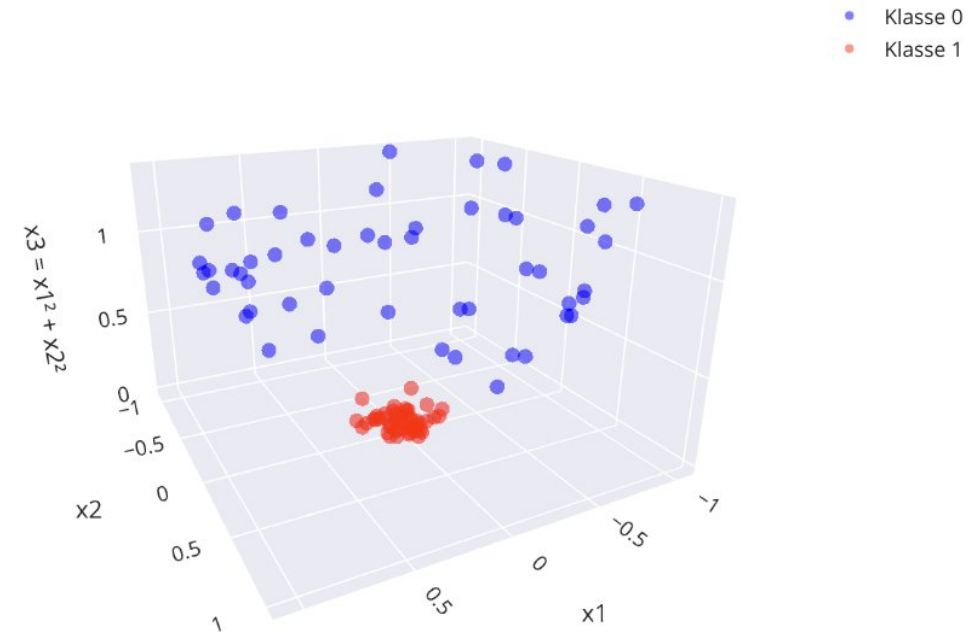
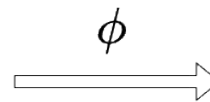
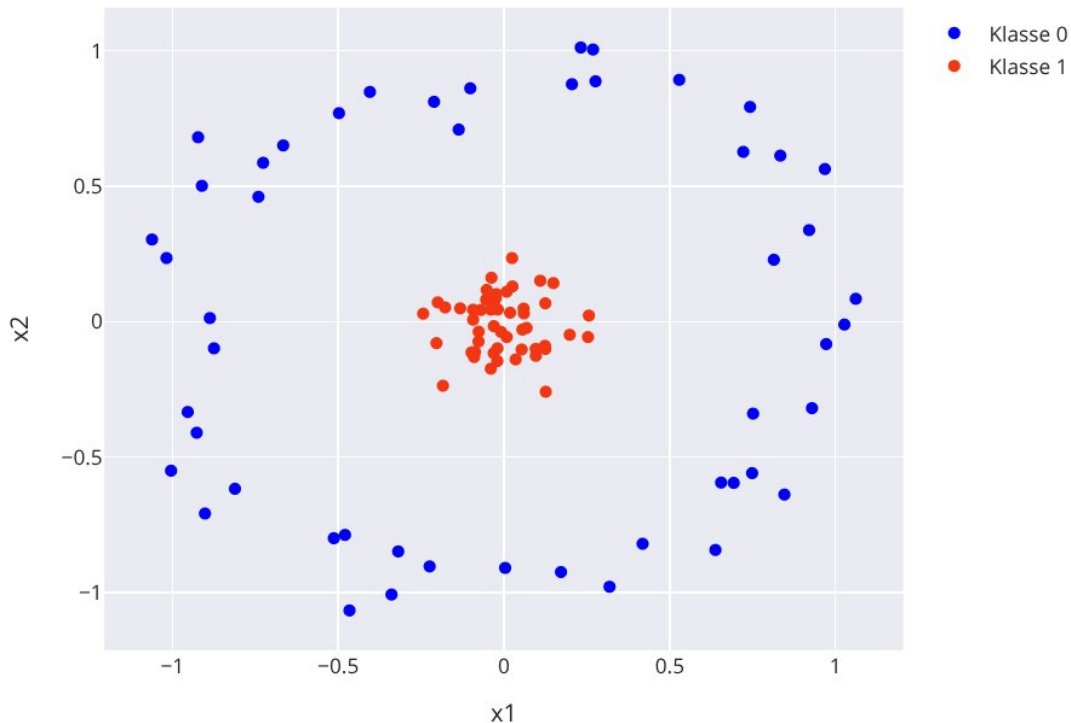


Beispiel: Kreisförmige Entscheidungsgrenze

Originaler Featureraum (x_1, x_2) : Lineare Grenze trennt die Klassen schlecht.

Idee: Neues Feature $x_3 = x_1^2 + x_2^2$ (quadratischer Abstand zum Ursprung)

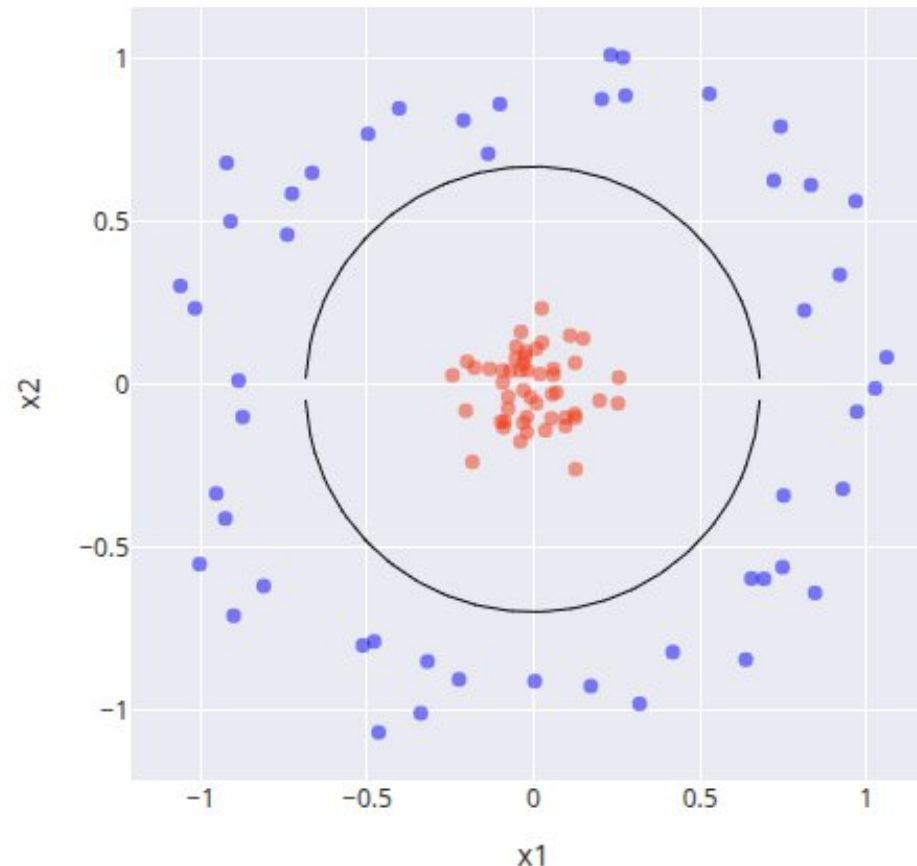
In 3D sind Punkte gut durch lineare Grenze (Ebene) trennbar!



Beispiel: Kreisförmige Entscheidungsgrenze



Rückprojection der Entscheidungsgrenze in den R^2 ergibt eine kreisförmige Entscheidungsgrenze:



Nichtlineare Features: Zusammenfassung

Prinzip: Erweiterung des Featurevektors durch nichtlineare Transformationen

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \rightarrow \phi(\mathbf{x}) = \begin{pmatrix} x_1 \\ x_2 \\ x_1^2 + x_2^2 \end{pmatrix}$$

- Logistisches Modell im erweiterten Raum ist **linear** in $\phi(\mathbf{x})$
- Entscheidungsgrenze im Originalraum kann **nichtlinear** sein
- Analogie zur polynomialen Regression (→ Kapitel 2)
- Risiko: **Overfitting** bei zu vielen Features (→ Kapitel Generalisierung)

Regularisierung

- Auch die logistische Regression kann bei vielen nichtlinearen Termen oder vielen Features (z.B. durch One-Hot Encoding mit sehr vielen Ausprägungen) **overfitten**
- Regularisierung wie bei linearer Regression durch ℓ_2 - oder ℓ_1 -Strafterm in der Loss-Funktion:

$$L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)] + \lambda \sum_{j=1}^p w_j^2$$

oder

$$L(\mathbf{w}) = \underbrace{-\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]}_{\text{Log-Loss Term}} + \underbrace{\lambda \sum_{j=1}^p |w_j|}_{\text{Strafterm}}$$

Lineare Modelle



	Lineare Regression	Logistische Regression
Aufgabe	Regression	Klassifizierung
Ausgabe	reelle Zahl	Wahrscheinlichkeit
Modellfunktion	$\mathbf{w}^T \mathbf{x}$	$\sigma(\mathbf{w}^T \mathbf{x})$
Verlustfunktion	MSE	Log-Loss
Training	analytisch / GD	Gradientenabstieg
Entscheidungsgrenze	—	linear

Zusammenfassung

- Logistische Regression erweitert lineare Regression auf **Klassifizierungsprobleme** (kategorische Zielvariable)
- **Sigmoid-Funktion** transformiert lineare Ausgabe in Wahrscheinlichkeiten; **One-vs-All** oder **Softmax** für mehrere Klassen
- Loss-Funktion: **Log-Loss** (= binäre Kreuzentropie)
- Prinzip identisch zur linearen Regression — andere Modell- und Verlustfunktion, gleicher Ansatz
- Training mit Gradientenabstieg
- Klassifikationsmodelle erlauben binäre Bewertung (richtig/falsch) → Metriken: Accuracy, Precision, Recall, ...
- **Schwellwert** beeinflusst Metriken und kann nach dem Training angepasst werden